



AULA 10

Métodos de classe

1. EXERCÍCIOS: EXIBIÇÃO DE DADOS DE LIVROS

- 1) Acesse o seu *github* e o repositório **pw2-play-livros**.
- 2) Clique no botão **Code** e copie o endereço do repositório.
- 3) Acesse o *Windows Explorer*, a unidade **D:** e a sua pasta de trabalho.
- 4) Abra um "cmd" e clone o projeto com:

```
git clone https://github.com/SEU-USUARIO/pw2-play-livros.git
```

- 5) Abra o projeto no *IntelliJ*.
- 6) Localize a classe `CarrinhoCompras` e renomeie para `TestaLivros`.
- 7) Inclua, na classe `TestaLivros`, após a linha `primeiroLivro.resumo`:

```
System.out.println("Título: " + primeiroLivro.titulo);  
System.out.println("Autor: " + primeiroLivro.autor);  
System.out.println("Narrador: " + primeiroLivro.narrador);  
System.out.println("Editora: " + primeiroLivro.editora);
```

- 8) Execute a classe para ver a saída.
- 9) Agora, a classe `TestaLivros` imprime os valores de título, autor, narrador, etc do `primeiroLivro`.

2. MÉTODOS DE CLASSE

Precisamos que todos os livros do sistema (inclusive os novos) mostrem seus dados na tela. Isso é algo que todo livro deve fazer, mas do jeito que implementamos, temos que repetir as mesmas linhas de código várias vezes:

```
System.out.println("Título: " + primeiroLivro.titulo);  
System.out.println("Autor: " + primeiroLivro.autor);  
System.out.println("Narrador: " + primeiroLivro.narrador);  
System.out.println("Editora: " + primeiroLivro.editora);
```

O que muda de um caso para outro é apenas o nome da variável — por exemplo, em vez de `primeiroLivro`, **usamos** `segundoLivro` ou `terceiroLivro`.

Imagine a seguinte situação: em breve, nosso cadastro terá cerca de 50 livros. Para cada livro, vamos precisar de 4 linhas de código para imprimir seus 4 atributos. Isso daria um total de 200 linhas de código muito parecidas — ou seja, praticamente repetidas.



Essa repetição de código sempre traz um problema chato: a manutenção fica difícil. Se um dia quisermos mostrar a duração do livro, vamos ter que alterar 50 trechos diferentes do nosso código, adicionando em cada um deles uma linha do tipo `System.out.println(livro.duracao)`.



Em vez de manter a lógica de exibição das informações do livro distribuída pelo código, podemos centralizar esse comportamento comum diretamente na classe `Livro`. Para isso, criamos um método.



Em Java, um método representa uma ação ou operação que o programa pode executar.

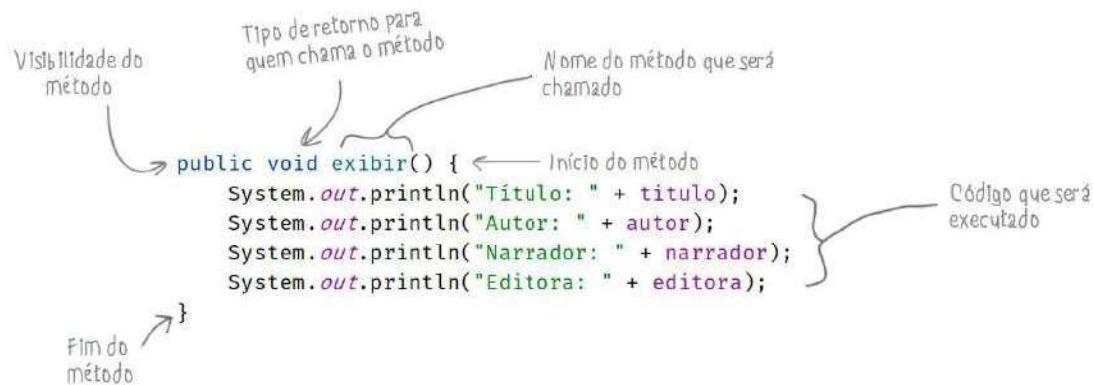
2.1. MÉTODO SEM RETORNO

Classes podem conter métodos que executam ações, mas não retornam nenhum valor. Eles realizam uma tarefa específica, porém não entregam um resultado de volta para quem os chamou.

Imagine que você peça a alguém para acender a luz. A pessoa vai até o interruptor e acende a luz, mas ela não te devolve nada — ela apenas realizou a tarefa. Não existe um "dado" ou "valor" resultante para você guardar ou usar depois.



Na programação, chamamos isso de método void. Utilize-o sempre que o objetivo do método for apenas executar uma ação (como exibir uma mensagem, salvar um arquivo, apagar um dado, enviar um e-mail, etc.), sem precisar devolver uma resposta.



```
public void exibir() {  
    System.out.println("Título: " + titulo);  
    System.out.println("Autor: " + autor);  
    System.out.println("Narrador: " + narrador);  
    System.out.println("Editora: " + editora);  
}
```

Visibilidade do método

Tipo de retorno para quem chama o método

Nome do método que será chamado

Início do método

Código que será executado

Fim do método

Nesse caso, o método não devolve nenhum valor ao final da execução, realizando as instruções que foram definidas dentro dele. Por isso, usamos o tipo de retorno `void`, que é a palavra reservada utilizada para indicar que um método executa uma ação, mas não retorna nenhum resultado.

Ao colocar esse comportamento dentro da classe `Livro`, perceberemos uma melhoria importante no código. O método `main` ficará com menos linhas e evitamos repetir várias vezes o mesmo trecho de código, aumentando o reaproveitamento e tornando a solução mais organizada.

```
primeiroLivro.resumo = "Explora como os hábitos ...";  
System.out.println("Título: " + primeiroLivro.titulo);  
System.out.println("Autor: " + primeiroLivro.autor);  
System.out.println("Narrador: " + primeiroLivro.narrador);  
System.out.println("Editora: " + primeiroLivro.editora);  
primeiroLivro.exibir();
```

Dessa forma, caso seja necessário modificar a maneira como os dados dos livros são exibidos, a mudança será feita em apenas um ponto: o método definido na classe `Livro`. Como esse comportamento está concentrado nesse local, qualquer ajuste realizado ali passa a valer automaticamente para todas as partes do sistema que utilizam esse método.

2.1.1. EXERCÍCIOS: MÉTODO VOID

- 1) Abra a classe `Livro` e defina o método `exibir()` abaixo:

```
public class Livro {
    //tipo nomeVariavel;
    public String titulo;
    public String autor;
    public String narrador;
    public String editora;
    public LocalDateTime duracao;
    public VersaoEnum versao;
    public double preco;
    public String resumo;

    public void exibir() {
        System.out.println("Titulo: " + titulo);
        System.out.println("Autor: " + autor);
        System.out.println("Narrador: " + narrador);
        System.out.println("Editora: " + editora);
    }
}
```

Digite "sout" e pressione ENTER

- 2) Agora que cada objeto do tipo `livro` possui o comportamento de exibir seus próprios detalhes, remova as linhas de código que realizavam essa tarefa e, no lugar delas, apenas chame o novo método.

```
primeiroLivro.resumo = "Explora como os hábitos ...";
System.out.println("Titulo: " + primeiroLivro.titulo);
System.out.println("Autor: " + primeiroLivro.autor);
System.out.println("Narrador: " + primeiroLivro.narrador);
System.out.println("Editora: " + primeiroLivro.editora);
primeiroLivro.exibir();
```

- 3) Execute a classe para ver a saída.
- 4) Repita a operação para os outros livros, usando o método `exibir()` e execute novamente a classe para ver a saída.
- 5) Agora, vamos mudar a forma como as informações dos livros aparecem na tela acrescentando um separador entre os livros. Será necessário alterar o método `exibir()` dentro da classe `Livro`.

```
public void exibir() {
    System.out.println("-----");
    System.out.println("Titulo: " + titulo);
    System.out.println("Autor: " + autor);
    System.out.println("Narrador: " + narrador);
    System.out.println("Editora: " + editora);
}
```

- 6) Execute a classe para ver a saída. Observe que a alteração foi feita em apenas um ponto e o ajuste passou a valer automaticamente para todas as partes do sistema que utilizam `exibir()`.
- 7) Execute os comandos para envio das alterações ao *github*:

```
git config --global user.name "nome1-nome2"
```

```
git config --global user.email "nome1-nome2@example.com"
```



```
git status
git add .
git commit -m "Criação do método exibir em Livro"
git push
```

2.1.2. EXERCÍCIO EXTRA: MÉTODO VOID

- 1) Na classe `Avaliacao`, crie um método que exiba os dados conforme abaixo:

Horácio Augusto da Silveira

5 estrelas Livro excelente

Avaliado em 2026-04-21T13:12

Livro excelente, muito bem escrito. Chegou no prazo e bem embalado

- 2) Crie, no pacote `test`, a classe `TestaAvaliacao` com um método `main`.
- 3) Instancie o usuário Horácio e associe à avaliação do exemplo acima.
- 4) Execute a classe para ver a saída.
- 5) Envie as alterações ao *github* com o texto *"Exercício extra com método void"*.

2.2. MÉTODO COM RETORNO

Nossa classe `Livro` possui o método `exibir()` que define nenhum retorno, onde usamos o `void`. Porém há situações em que precisamos retornar algo como, por exemplo, quando interagimos com o dispositivo *Alexa*.

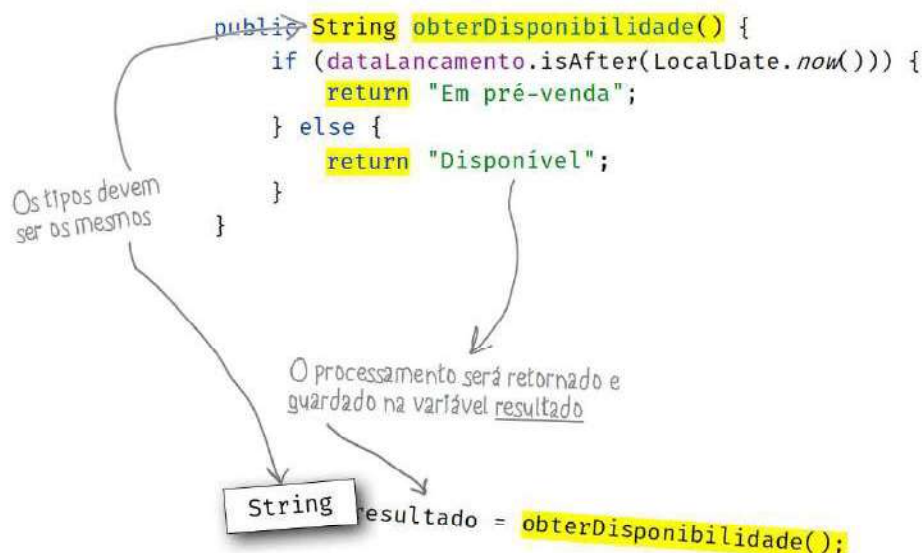


PODEMOS RECUPERAR INFORMAÇÕES EM UM MÉTODO.

A imagem mostra uma situação simples: uma pessoa faz uma pergunta, recebe uma resposta e, a partir disso, toma uma decisão (pensar em usar um casaco).

Podemos relacionar isso diretamente com o conceito de **métodos com retorno**. Em programação, quando chamamos um método, estamos basicamente “fazendo uma pergunta” ao sistema. Esse método executa alguma lógica e devolve um resultado — assim como a Alexa faz na imagem.

Imagine que, em nosso sistema da livraria, queremos classificar a disponibilidade de um livro com base em sua data de lançamento. O método realizará uma série de verificações condicionais utilizando o atributo `dataLancamento`, tendo uma `String` como retorno. Para retornar um valor, utilizamos a palavra reservada `return`:



Agora que `Livro` ganhou esse novo método, vamos aproveitá-lo dentro do método `exibir()` da própria classe, mostrando a disponibilidade daquele objeto. Uma forma de fazer isso seria:

```
public void exibir() {  
    System.out.println("_____");  
    System.out.println("Título: " + titulo);  
    System.out.println("Autor: " + autor);  
    System.out.println("Narrador: " + narrador);  
    System.out.println("Editora: " + editora);  
    System.out.println("Disponibilidade: " + obterDisponibilidade());  
}
```

Um método sempre tem que definir o que retorna, nem que defina que não há retorno, como no exemplo anterior onde estávamos usando o `void`.

2.2.1. EXERCÍCIOS: MÉTODO COM RETORNO

- 1) Abra a classe `Avaliacao` e codifique o método `obterDataPorExtenso()` que retornará uma `String` com a data de avaliação no formato *10 de janeiro de 2026*.

```
public String obterDataPorExtenso() {  
}
```

- 2) Para retornar a data por extenso, usaremos o formato `"d 'de' MMMM 'de' yyyy"` definido na classe `DateTimeFormatter`.

```
public String obterDataPorExtenso() {  
    DateTimeFormatter formatador = DateTimeFormatter.ofPattern("d 'de' MMMM 'de' yyyy");  
}
```

- 3) Utilizaremos uma `String` para receber o resultado do método `format` do objeto `formatador` declarado acima.

```
public String obterDataPorExtenso() {  
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("d 'de' MMMM 'de' yyyy");  
    String resultado = formatador.format(data);  
}
```

- 4) Por fim, retorne o resultado:

```
public String obterDataPorExtenso() {  
    DateTimeFormatter formatador = DateTimeFormatter.ofPattern("d 'de' MMMM 'de' yyyy");  
    String resultado = formatador.format(data);  
    return resultado;  
}
```

- 5) Altere o método `exibir()` de `Avaliacao` para imprimir, na linha *"Avaliado em "*, o resultado do método acima:

```
public void exibir() {  
    System.out.println(" ");  
    System.out.println(usuario.nome);  
    System.out.println(estrelas + " estrelas " + titulo);  
    System.out.println("Avaliado em " + obterDataPorExtenso());  
    System.out.println(comentario);  
}
```

- 6) Execute a classe `TestaAvaliacao` para ver a saída.
- 7) Envie as alterações ao *github* com o texto *"Criação do método obterDataPorExtenso em Avaliacao"*.

2.2.2. EXERCÍCIO EXTRA: MÉTODO COM RETORNO

- 1) Abra a classe `Livro` e defina o atributo `dataLancamento` do tipo `LocalDate`.
- 2) Agora, codifique o método `obterDisponibilidade()` que, inicialmente, obtém a data atual por meio da classe `LocalDate`. Em seguida, o método realizará uma série de verificações condicionais utilizando a variável `dataLancamento`:

- Caso a data de lançamento não esteja definida (ou seja, seja nula), o método retorna "Indisponível", indicando que não há informações suficientes sobre o livro.
 - Se a data de lançamento for posterior à data atual, o livro ainda não foi lançado, sendo classificado como "Em pré-venda".
 - Se a data de lançamento estiver dentro dos últimos 30 dias em relação à data atual, o livro é considerado um "Lançamento recente".
 - Em qualquer outro caso, entende-se que o livro já foi lançado há mais tempo, sendo classificado como "Disponível".
- 3) Altere o método `exibir()` para que imprima a String retornada por `obterDisponibilidade()`.
 - 4) Na classe `TestaLivros`, inclua a `dataLancamento` para os livros informados.
 - 5) Execute a classe para ver a saída.
 - 6) Crie a enumeração `DisponibilidadeEnum` no pacote `enums`.



- 7) Altere o método `obterDisponibilidade()` de `Livro` para que retorne, no lugar de `String`, a `DisponibilidadeEnum`.
- 8) Execute a classe `TestaLivros` para ver a saída.
- 9) Envie as alterações ao *github* com o texto "Criação do método `obterDisponibilidade` em `Livro`".

2.3. PARÂMETROS EM MÉTODOS

Os métodos podem receber valores externos, chamados parâmetros, que servem como entradas para a execução da ação. Isso permite que o mesmo método funcione de formas diferentes conforme os valores passados.



**SE UM MÉTODO RECEBE
PARÂMETRO(S), VOCÊ DEVE
PASSAR ALGUMA COISA
PARA ELE!**

A imagem representa um método que cria (fabrica) um objeto do tipo `Pizza`, recebendo como entrada um parâmetro `sabor` que funciona como uma instrução para o método. Sem esse valor, o método não saberia qual pizza criar.

2.3.1. EXERCÍCIOS: MÉTODO COM PARÂMETRO

- 1) Abra a classe `Avaliacao` e defina o método `adicionar`.

```
public void adicionar(int nota) {
    this.estrelas = nota;
}
```

- 2) Abra a classe `TestaAvaliacao`, substitua a linha que altera a quantidade de estrelas pelo método `adicionar` com a nota desejada.

```
avaliacaoHoracio.estrelas = 5;
avaliacaoHoracio.adicionar(5);
```

- 3) Execute a classe para ver a saída.

Em Java, a palavra-chave `this` é uma referência ao próprio objeto atual, sendo usado principalmente para acessar atributos do próprio objeto, diferenciando-os de parâmetros com o mesmo nome.

2.4. SOBRECARGA DE MÉTODO

A sobrecarga de métodos em Java é um recurso que permite criar vários métodos com o mesmo nome, mas com parâmetros diferentes dentro da mesma classe. A sobrecarga oferece flexibilidade, pois dependendo da situação, o usuário pode fornecer mais ou menos informações.

Para que haja sobrecarga, os métodos precisam diferir em pelo menos um destes pontos:

- Número de parâmetros
- Tipo dos parâmetros
- Ordem dos parâmetros (se forem de tipos diferentes)

O tipo de retorno sozinho **NÃO** é suficiente para diferenciar métodos.

Por exemplo, na classe `Avaliacao`, o método `adicionar` foi definido mais de uma vez, com diferentes listas de parâmetros:

C Avaliacao	
o	estrelas: int
o	titulo: String
o	data: LocalDateTime
o	comentario: String
o	usuario: Usuario
o	aprovado: boolean
●	exibir(): void
●	aprovar(): void
●	gerarEstrelinhas(): String
●	adicionar(nota): void
●	adicionar(nota, comentario): void
●	adicionar(nota, titulo, comentario): void

A sobrecarga do método `adicionar` permite que um mesmo comportamento (adicionar uma avaliação) seja realizado de diferentes formas:

- Apenas a nota
- Nota e comentário
- Nota, título e comentário

Assim, o método se adapta ao contexto sem exigir sempre todos os dados.

2.4.1. EXERCÍCIOS: SOBRECARGA DE MÉTODO

- 1) Abra a classe `Avaliacao` e adicione duas novas versões do método `adicionar`.

```
public void adicionar(int nota) {  
    this.estrelas = nota;  
}  
  
public void adicionar(int nota, String comentario) {  
    this.estrelas = nota;  
    this.comentario = comentario;  
}  
  
public void adicionar(int nota, String titulo, String comentario) {  
    this.estrelas = nota;  
    this.titulo = titulo;  
    this.comentario = comentario;  
}
```

- 2) Abra a classe `TestaAvaliacao`, instancie o novo usuário Renata e associe à uma nova avaliação utilizando o método `adicionar(3, "Bom", "Para mim que tenho filho maior que 5 anos, não achei interessante.")` para incluir os dados da avaliação abaixo:

Renata Maciel

3 estrelas Bom

Avaliado em 23 de janeiro de 2025

Para mim que tenho filho maior que 5 anos, não achei interessante.

- 3) Execute a classe para ver a saída.
- 4) Envie as alterações ao *github*:

```
git config --global user.name "nome1-nome2"  
git config --global user.email "nome1-nome2@example.com"  
git status  
git add .  
git commit -m "Criação de métodos adicionar() em Avaliacao"  
git push
```

AULA 11

Construtores de Classe

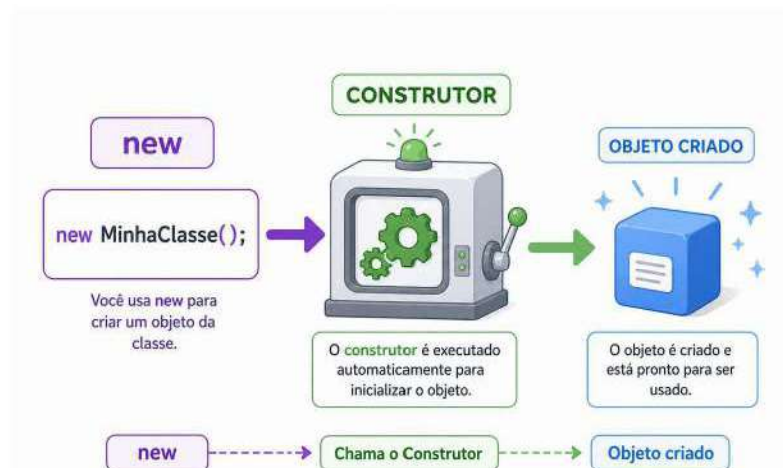
1. CONSTRUTORES DE CLASSE

Em Java, objetos são criados a partir de classes utilizando a palavra-chave `new`. A criação de um objeto segue, em geral, este formato:

```
Usuario horacio = new Usuario();
```

Parece que estamos chamando um método chamado Usuario(), por causa dos parênteses.

Um construtor contém o código que é executado quando você instancia um objeto, ou seja, o código que é executado quando você usa `new` em um tipo de classe.



Podemos escrever um construtor para nossas classes, mas se não o fizermos, o compilador escreverá um para nós. Veja como é o construtor padrão do compilador:

```
public Usuario() {  
    // código do construtor  
}
```

Se fosse um método, precisaríamos de um tipo de retorno entre "public" e "Usuario".

Seu nome é o mesmo nome da classe

A principal característica de um construtor é que ele é executado antes que o objeto possa ser atribuído a uma variável, sendo muito comum a definição de valores iniciais para seus atributos, permitindo

que os objetos já nasçam configurados, evitando a criação de instâncias “vazias” que precisam ser preenchidas atributo por atributo.

Por exemplo, na classe `Usuario`, o construtor pode ser usado para garantir que um usuário já seja criado com informações essenciais, como o nome. Com isso, garantimos que o objeto já seja criado em um estado válido e evitamos esquecer de inicializar atributos importantes.

```
public class Usuario {
    public String foto;
    public String nome;

    public Usuario(String nome) {
        this.nome = nome;
    }
}

public class TestaUsuario {

    static void main() {
        Usuario horacio = new Usuario("Horácio Augusto");
    }
}
```

Parâmetro nome para o construtor de Usuario

Passando o valor para o construtor

1.1. EXERCÍCIOS: CONSTRUTOR

- 1) Abra a classe `Usuario` e defina o construtor abaixo que receberá, obrigatoriamente, o nome do usuário:

```
public class Usuario {
    public String foto;
    public String nome;

    public Usuario(String nome) {
        this.nome = nome;
    }
}
```

- 2) Agora, abra a classe `TestaAvaliacao` e observe que, na instanciação do usuário, é obrigatório informar o nome do usuário no construtor.

```
Usuario horacio = new Usuario("Horácio Augusto da Silveira");
horacio.nome = "Horácio Augusto da Silveira";
```

- 3) Execute a classe para ver a saída.
- 4) Agora, com o uso do construtor em `Usuario`, podemos simplificar a atribuição de um usuário na avaliação. Faça:

```
static void main() {
    Usuario horacio = new Usuario("Horácio Augusto da Silveira");

    Avaliacao avaliacaoHoracio = new Avaliacao();
    avaliacaoHoracio.usuario = new Usuario("Horácio Augusto da Silveira");
}
```




7) Execute os comandos para envio das alterações ao *github*:

```
git config --global user.name "nome1-nome2"  
git config --global user.email "nome1-nome2@example.com"  
git status  
git add .  
git commit -m "Criação de construtor em Usuario"  
git push
```